LEARNING THROUGH RECONSTRUCTION · DELIVERABLE 4

Data Flow

The write path, the read path, the admission decision, and the deletion cascade — step by step.

System Conversational Memory Intelligence System
Companions architecture.pdf · api_contracts.md · data_model.md
Author Dheeraj Pranav
Date 22 July 2026

processing · admission gate (TB-1) · transaction · trace / audit emit (C9)

1. Write path — `POST /memory/admit`

Untrusted turns become typed, screened, stored facts. Every stage emits a trace event (C9); the commit and its embedding are one transaction so the index never lags the source of truth.

1	**Agent → API**	TENANT FROM AUTH CONTEXT — NO TENANT_ID IN BODY `admit(account, user, session, turns[])`
2	**Extractor**	turns → candidate typed claims `{type, subject, content, confidence, observed_at}` C1 · C3 · LLM · FIXES F7 (CHUNK→FACT)
3	**PII Gate**	Presidio deterministic-first screen; ineligible candidates **blocked before storage** C2 · F11 · HARD GATE — NO OUTPUT-PATH REDACTION (ADR-003)
4	**Op-Selector**	per surviving candidate: **store | update | discard** vs current facts on (account, subject) C2 · C7 · F8 · MEM0-STYLE, PROTOTYPE — SEE §3
5	**Committer (txn)**	**store**: INSERT current fact · **update**: invalidate prior (valid_to, invalidated_at/by, status=superseded) + INSERT new · **discard**: log only C3 · C7 · F4 · SUPERSESSION ON THE WRITE PATH (ADR-001)
6	**Embedder (same txn)**	embed committed facts → `memory_embedding` C4 · DERIVED PROJECTION; DELETE LATER CASCADES HERE
7	**Trace**	emit extract / pii / op-select / commit events under one `trace_id` C9 · P7
8	**API → Agent**	`{admitted[], updated[], discarded[], pii_blocked[], trace_id}`

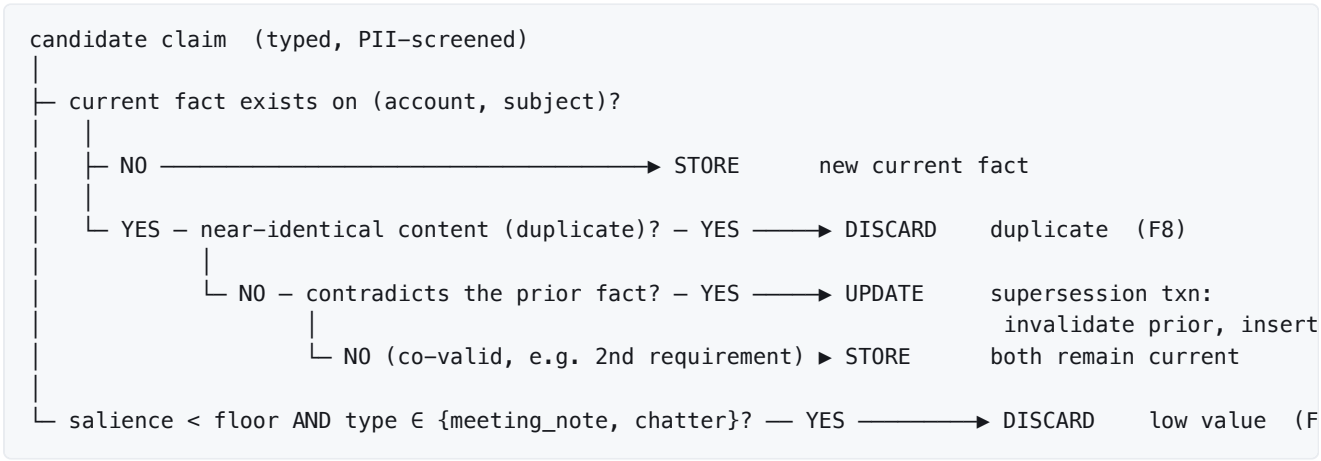
2. Read path — `POST /memory/retrieve`

Isolation is applied at the index scope (step 2), **before** ranking — so a foreign near-duplicate is never even a candidate, which is why a better embedder cannot leak it (the D3 7/11 finding).

1	**Agent → API**	TENANT FROM AUTH CONTEXT `retrieve(account, user, query, token_budget=2000, k=12)`
2	**Retriever**	**tenant+account-scoped** ANN over `memory_embedding`, **currently-valid only** → top-k candidates C5 · C8 · F10 · ISOLATION APPLIED HERE (RLS + REPO SCOPE)
3	**Ranker**	score each candidate = weighted(relevance, recency, importance-by-type, confidence) C5 · HAND-SPECIFIED WEIGHTS · CONFLICT ALREADY RESOLVED AT WRITE TIME
4	**Constructor**	add in rank order until budget binds; position-aware ordering; if top score < abstain floor → **abstain** (empty set) C6 · F2/F3 · COLD-START FIX (D3: BASELINE FAILED 2/2)
5	**Audit (fail-closed)**	write `access_log` row (returned ids); **no result returns without an audit row** C8 · C9 · THREAT R5
6	**API → Agent**	`{injected[] (ordered), abstained, dropped_for_budget[], used_tokens, trace_id}`

3. Op-selector decision tree (step 1.4)

The admission decision that keeps the index clean (F8) and drives supersession (F4). Marked *prototype*: the per-candidate LLM comparison is the ≤15%-cost-budget risk (threat R6), measured in D6.



4. Deletion cascade & the F9 window (`POST /deletion`)

The single-store choice (ADR-005) makes erasure authoritative: source and index die in the same transaction, so the typical consistency window is a transaction, not a cross-store gap.

t0	API	create `deletion_request` (status=pending, requested_at) C7 · SCOPE = SUBJECT|ACCOUNT|USER|TENANT
t1	Txn BEGIN	resolve targets via `memory_source` provenance (find every copy)
t2	Delete facts	DELETE matching `memory` rows
t3	Cascade index	FK `ON DELETE CASCADE` removes `memory_embedding` C4 · F9 · SAME TXN — NO LAG
t4	Txn COMMIT	set `completed_at` , status=completed
t5	API → caller	`{deleted_count, window_ms = completed_at − requested_at}` — the guarantee is **measured**, not assumed
t6	Backstop (async)	backup erasure within ≤24 h (ADR-004) — the honest edge; live store is synchronous, backups within the backstop THREAT R3

5. Consolidation — cite, don't replace (lifecycle job)

The highest-risk flow (first_principles §5): three capabilities cooperate. The guard is structural — a rollup memory **links** to its sources; the sources stay retrievable and deletable.

```
sources: m_a, m_b, m_c  (related current facts)
        │
        ▼  consolidate
rollup memory R  ──cites──▶ consolidation_edge(R→m_a), (R→m_b), (R→m_c)
        │
        ├─ m_a, m_b, m_c REMAIN current & retrievable   (no loss → avoids F5)
        └─ deleting m_b later stays authoritative        (not hidden inside R → avoids F6/F9)
```

AI assistance disclosure. Claude helped render these flows. The sequencing, the op-selector logic, the isolation-before-ranking ordering, and the deletion-cascade design are mine and are what I will defend in review. Every step cites the capability, premise, or failure it serves.